

DOCUMENTATION TECHNIQUE · MICHELIN AURORA

AURORA

Guide d'initialisation environnement DEV

Lancer le projet complet en local — de zéro à l'app qui tourne —
en moins de 10 minutes.



Docker Compose



Nuxt 4 & Symfony 7



PostgreSQL 16

 go-task



Python 3



OUTILS OBLIGATOIRES



Git

Toute version récente

Pour cloner le dépôt GitHub et gérer les versions du projet.



Docker Desktop

v4.x ou supérieur

Fait tourner les 4 conteneurs (back, front, base de données, PgAdmin). Inclut Docker Compose v2.



go-task

v3.x — taskfile.dev

Gestionnaire de tâches qui simplifie toutes les commandes. Remplace les longues lignes `docker compose exec ...`



Python 3 (optionnel)

v3.9 ou supérieur

Uniquement pour le simulateur IoT ESP32. Pas nécessaire si vous ne testez pas la démo télémétrie live.

VÉRIFIER L'INSTALLATION

```
$ git --version          # git version 2.x.x
$ docker --version       # Docker version 26.x.x
$ docker compose version # Docker Compose version v2.x.x
$ task --version         # Task version: v3.x.x
$ python3 --version      # Python 3.x.x
```

INSTALLER GO-TASK

```
# macOS
$ brew install go-task

# Linux
$ sh -c "$(curl --location https://taskfile.dev/install.sh)" -- -d -b /usr/local/bin

# Windows (Scoop)
$ scoop install task
```



Docker Desktop doit être démarré avant toute commande `task` ou `docker compose`. Ouvrez l'application et attendez que le moteur soit en état "Running" (icône verte dans la barre système).

CONTENEURS QUI SERONT LANCÉS

CONTENEUR	RÔLE	PORT LOCAL
back	API Symfony 7 + Nginx — toutes les routes <code>/api/...</code>	8081
front	Application Nuxt 4 — interface utilisateur PWA	3001
db	Base de données PostgreSQL 16	5432
pgadmin	Interface web d'administration de la base de données	5050



ÉTAPE 1 — CLONER LE DÉPÔT



Cloner le projet depuis GitHub

Ouvrez un terminal, placez-vous dans le dossier souhaité et clonez le dépôt.

```
$ git clone https://github.com/Ekin0ox/michelin-aurora.git
$ cd michelin-aurora
```

ÉTAPE 2 — FICHIER D'ENVIRONNEMENT



Copier `.env.example` → `.env`

Le fichier fonctionne tel quel pour un environnement local. Aucune modification nécessaire.

```
$ cp .env.example .env # valeurs par défaut prêtes à l'emploi
```

Contenu du `.env` généré :

```
APP_ENV=dev
APP_SECRET=changeme_in_production

POSTGRES_DB=aurora      POSTGRES_USER=aurora
POSTGRES_PASSWORD=aurora
DATABASE_URL=postgresql://aurora:aurora@db:5432/aurora?serverVersion=16

PGADMIN_EMAIL=admin@michelin-aurora.dev
PGADMIN_PASSWORD=aurora

BACK_PORT=8081      FRONT_PORT=3001
Nuxt_PUBLIC_API_BASE=http://localhost:8081
```



Conflit de port ? Si le port `8081` ou `3001` est occupé sur votre machine, modifiez `BACK_PORT` et/ou `FRONT_PORT` dans `.env`, puis mettez à jour `Nuxt_PUBLIC_API_BASE` en conséquence.

ÉTAPE 3 — DÉMARRER LES CONTENEURS



Lancer toute la stack Docker

Construit les images (première fois) et démarre les 4 conteneurs en arrière-plan.

```
$ task up:build # première fois – build complet
$ task up      # fois suivantes – démarrage rapide (sans rebuild)
```



Vérifier que tous les conteneurs sont actifs

```
$ task ps

michelin-aurora-back-1    Up           0.0.0.0:8081->8080/tcp
michelin-aurora-front-1  Up           0.0.0.0:3001->3000/tcp
michelin-aurora-db-1     Up (healthy) 0.0.0.0:5432->5432/tcp
michelin-aurora-pgadmin-1 Up           0.0.0.0:5050->80/tcp
```



Première fois : le build peut prendre **2 à 5 minutes** (téléchargement des images, Composer, npm). Les fois suivantes, le démarrage prend moins de 10 secondes.



ÉTAPE 4 — INITIALISER LA BASE DE DONNÉES



Option A — Initialisation complète en une commande

Recommandé

Enchaîne automatiquement : suppression → création → migrations → chargement des fixtures.

```
$ task db:reset
```



Option B — Étapes manuelles

```
# 1. Appliquer les migrations Doctrine (crée toutes les tables)
$ task migrate

# 2. Charger les données de test (utilisateurs, pneus, routes...)
$ task fixtures
```

CONTENU DES FIXTURES

DONNÉES	CONTENU CHARGÉ
Utilisateurs	Plusieurs comptes de test avec profils variés (Route, Gravel, VTT, VAE)
Pneus Michelin	Catalogue complet (Power Road, Power Gravel, Wild Enduro...) avec scores
Itinéraires	Routes avec scores Michelin, sécurité, plaisir et pneu recommandé
Événements	Courses et events cyclistes avec discipline et distance
Récompenses	Catalogue d'échanges avec codes réduction Michelin

COMMANDES COURANTES

```
# Réinitialiser complètement (efface toutes les données)
$ task db:reset

# Recharger les fixtures sans recréer la base
$ task fixtures

# Appliquer une nouvelle migration après un git pull
$ task migrate
```

EXPLORER LA BASE — PGADMIN



Accès PgAdmin via navigateur

URL : `http://localhost:5050`

Email : `admin@michelin-aurora.dev` · Mot de passe : `aurora`

Ajouter un serveur : hôte `db` · port `5432` · utilisateur `aurora` · mdp `aurora`



`task down:volumes` supprime définitivement la base de données. Utilisez-la uniquement pour repartir de zéro.
Pour un simple redémarrage, préférez `task down` puis `task up`.



🔗 URLS DISPONIBLES APRÈS DÉMARRAGE

🔗 URL	SERVICE	DESCRIPTION
localhost:3001	Frontend Aurora Nuxt	Interface principale de l'application PWA
localhost:8081	API Symfony REST	Toutes les routes <code>/api/...</code> du backend
localhost:8081/api/health	Health check	Retourne <code>{"status": "ok"}</code> si le back est actif
localhost:5050	PgAdmin	Interface web de gestion PostgreSQL

🔧 VÉRIFICATION RAPIDE



Test backend — Health check

```
$ curl http://localhost:8081/api/health
{"status": "ok", "env": "dev"}
```



Compte de test (chargé par les fixtures)

```
Email      : test@aurora.dev
Password   : password
```

📱 ACCÈS MOBILE — TESTER LA PWA



Trouver l'IP locale de votre machine

```
# macOS
$ ipconfig getifaddr en0

# Linux
$ hostname -I | awk '{print $1}'

# Windows
> ipconfig # → section Wi-Fi → Adresse IPv4
```



Accéder et installer la PWA sur iPhone

Sur votre téléphone (même réseau Wi-Fi), ouvrez Safari et naviguez vers :

```
http://[VOTRE-IP]:3001 # ex: http://192.168.1.42:3001
```

iOS : menu Partage → "Sur l'écran d'accueil" pour installer la PWA.

🧪 LANCER LES TESTS

```
$ task test           # tous les tests (PHPUnit + Vitest)
$ task back:test      # PHPUnit uniquement (backend Symfony)
$ task front:test     # Vitest unitaires (composables Nuxt)
$ task front:test:e2e # Playwright E2E (nécessite le serveur actif)
```



☰ LANCER LE PROJET EN 5 COMMANDES

1

Cloner le dépôt

```
git clone https://github.com/Ekin0ox/michelin-aurora.git && cd michelin-aurora
```

2

Créer le fichier d'environnement

```
cp .env.example .env
```

3

Construire et démarrer les conteneurs

```
task up:build
```

2-5 min la première fois (images Docker, Composer, npm)

4

Initialiser la base de données

```
task db:reset
```

Migrations + fixtures en une seule commande

5

Ouvrir l'application dans le navigateur

```
http://localhost:3001
```

API : <http://localhost:8081/api/health> · PgAdmin : <http://localhost:5050>

🔗 SIMULATEUR IOT ESP32 (DÉMO TÉLÉMÉTRIE)

À quoi ça sert ? Le simulateur imite un capteur pneu connecté qui envoie en temps réel la pression avant/arrière et la vitesse — sans matériel IoT physique.

```
$ task sim # ou : python3 simulator/esp32_sim.py

→ Ride démo créé : 3f8a9b...
→ Telemetry push [OK] pressure=2.6/2.8 speed=24.3 km/h
⚠ ALERT triggered - pressure front = 1.8 bar (seuil : 2.0)
```

☰ COMMANDES DE GESTION COURANTES

- | | |
|--|---|
| task up
Démarre la stack (sans rebuild) | task down
Arrête la stack (conserve les données) |
| task logs -- back
Logs du conteneur back en temps réel | task shell:back
Ouvre un terminal dans le conteneur Symfony |
| task back:cc
Vide le cache Symfony | task down:volumes
⚠ Supprime stack + base de données |



Stack opérationnelle · back : 8081 · front : 3001 · db : 5432 · pgadmin : 5050
Structure : apps/back/ (Symfony) · apps/front/ (Nuxt) · simulator/ (IoT) · Taskfile.yml

